

Towards Building Post-Quantum Secure Ethereum

Howell Liu, Zhipeng Wang, William Knottenbelt
Imperial College London, United Kingdom

Abstract—Public blockchains, such as Ethereum and Bitcoin, use the Elliptic Curve Digital Signature Algorithm (ECDSA), specifically `Secp256k1`, to enable users to authenticate transactions. However, this cryptographic system is relatively vulnerable to quantum attacks, which may become truly feasible in the next few years through quantum computing advances. Existing studies recognize the importance of post-quantum secure signatures, but the actual implementation and transition of post-quantum secure blockchains still require further investigation.

In this work, we leverage `pk-FALCON512`, a lattice-based and post-quantum secure signature scheme, to replace `Secp256k1` in the Ethereum execution client. We analyze the performance and storage implications of this replacement. Our evaluation results show that `pk-FALCON512` is approximately $23,000\times$, $28\times$, and $3\times$ times slower than `Secp256k1` in key generation, signature generation, and signature verification, respectively. Specifically, signature verification, critical for on-chain operations, increases from $24\mu s$ to $74\mu s$, and this indicates an acceptable trade-off for adopting post-quantum secure signatures. In terms of storage, `pk-FALCON512` needs $15\times$ more space with cryptographic elements, which might increase the average block size by approximately $3.85\times$. These results show feasibility in practice for integrating post-quantum cryptography schemes into Ethereum for further research.

I. INTRODUCTION

Many public blockchains, such as Bitcoin and Ethereum, rely on the Elliptic Curve Digital Signature Algorithm (ECDSA) [1], [2] for transaction authentication. ECDSA has its security foundations in the large mathematical complexity of the elliptic curve discrete logarithm problem [3]. For classical computers, it is computationally infeasible with large finite fields. Nevertheless, quantum algorithms, such as Shor's algorithm [4], can solve the ECDLP efficiently, making elliptic curve-based cryptosystems insecure. With quantum computing expected to become widely accessible in the near future [5], in 2024, the National Institute of Standards and Technology (NIST) has called for a migration to post-quantum cryptography through its initiative, *Migration to Post-Quantum Cryptography: Considerations for Transitioning to Post-Quantum Cryptographic Algorithms* [6]. This highlights the need to develop cryptographic systems that are resistant to quantum attacks. Following this initiative, large technology companies, such as Apple [7] and Zoom [8], have already started including post-quantum cryptography in their products. Ethereum core developers have also acknowledged this trend and have proposed integrating quantum resistance in future upgrades [9].

Although existing studies [10], [11], [12] explore post-quantum cryptography's theoretical compatibility with blockchain systems, certain research gaps remain. For instance, many existing works focused on investigating the

signature candidates to replace existing post-quantum cryptographic schemes in blockchains, but they have not proposed concrete implementation details. We need more exploration of the *practical* implications of replacing cryptographic primitives in specific blockchain protocols that are highly performant.

In this work, we address these gaps by providing a detailed examination of transitioning public blockchains to post-quantum cryptographic schemes, focusing on the Ethereum execution client protocol as a case study. Specifically, we propose a practical implementation by replacing the Ethereum ECDSA signature with FALCON [13], [14], a lattice-based post-quantum secure signature scheme.

Contributions. Our contributions are summarized as follows:

- We show the importance of public key recovery when integrating post-quantum signatures into Ethereum. Public key recovery in Ethereum can minimize on-chain storage of cryptographic elements. Although public key recovery has been theoretically described for schemes such as FALCON [14], its practical implementation remains unexplored. We develop a concrete implementation of public key recovery for FALCON in Python and Rust, enabling empirical analysis of its performance within the Ethereum client.
- We quantify the performance changes and trade-offs associated with replacing Ethereum's existing signature scheme, `Secp256k1`, with a post-quantum alternative, `pk-FALCON512`, a 512 byte public key recovery implementation of FALCON. Our findings show important performance bottlenecks, such as a $23,000\times$, a $28\times$, and a $3\times$ slower speed for key generation, signature generation, and signature verification respectively. We also show a $15\times$ increase in the storage required for cryptography elements and a $3.85\times$ increase in the average block size. These results indicate the need for further research to optimize post-quantum cryptographic schemes for blockchain systems to balance security, performance, and scalability requirements.

II. BACKGROUND

A. Digital Signatures in Ethereum Transactions

A digital signature scheme (DSS) is a cryptographic protocol for verifying the authenticity and integrity of a message transmitted over non-secure channels [15], using a private key to sign messages and a public key to verify signatures. Ethereum uses DSSs to ensure transaction authenticity while also protecting the secrecy of the sender's private key. The execution layer and the consensus layer rely on elliptic-curve cryptography to secure the system. The execution layer uses

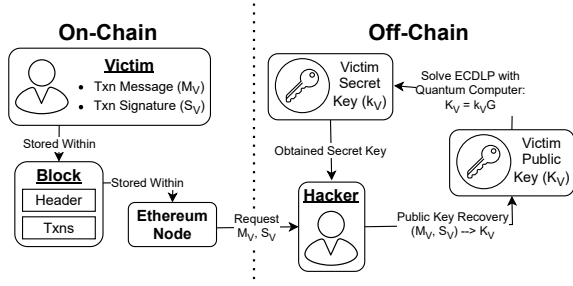


Fig. 1. Diagram showing how an attacker can obtain a victim's secret key in a system using the elliptic curve digital signature algorithm.

ECDSA, especially *Secp256k1*, for authenticating transactions [16]. Meanwhile, the consensus layer uses the BLS digital signature algorithm to secure the identities of participants in the consensus layer and to ensure secure communication. In this work, we focus on the post-quantum security of the execution layer.

B. Post-Quantum Secure Digital Signatures

Elliptic curve cryptosystems, which rely on the computational difficulty of the ECDLP [3], are secure against classical computers, but are vulnerable to quantum attacks. While the fastest classical algorithm, Pollard-Rho, has a time complexity of $O(\sqrt{N})$ [17], quantum computers can solve ECDLP efficiently using Shor's algorithm, with a time complexity of $O((\log N)^3)$ [4], rendering elliptic curve cryptography insecure. This poses a critical threat to blockchains such as Ethereum, where hackers could exploit quantum capabilities to derive private keys from publicly stored data and forge signatures (see Figure 1).

To address this, NIST initiated a search in 2016 for post-quantum cryptosystems, selecting CRYSTALS-Dilithium, SPHINCS+, and FALCON as digital signature scheme winners in 2022 [18], [19]. Among these, FALCON, based on lattice cryptography, is ideal for blockchains due to its compact keys, small storage requirements, and fast speed in signature generation and verification, making it well-suited for performance-critical and storage-constrained systems.

III. METHODOLOGY FOR BUILDING FALCON-BASED ETHEREUM CLIENT

A. Theoretical Preparation

FALCON stands for Fast-Fourier Lattice-based Compact signature over NTRU [14]. FALCON is ideal for blockchains compared to the other post-quantum cryptography scheme options because it requires the least storage for its private keys and signatures. FALCON is built on lattice cryptography, and it relies on the difficulty of the shortest integer solutions (SIS) problem, which is NP-hard [20] and does not have a quantum computer algorithm that can solve it efficiently. The SIS problem involves finding a non-zero vector x such that $0 < \|x\| \leq \beta$ and $Ax = 0$ in $\mathbb{Z}_q$ in the q -ary lattice, A_q [20]. $\|\cdot\|$ denotes the L_2 norm.

1) *Key Generation*: FALCON uses unique NTRU lattices, which are special lattices constructed on a polynomial ring $R = \mathbb{Z}[x]/\phi$ where $\phi = x^n + 1$. The NTRU lattices used for FALCON are composed of four polynomials, (f, g, F, G) , that satisfy $fG - gF = q \pmod{\phi}$, where q is a prime, typically 12289. Those polynomials are selected to be short, meaning that they will be able to generate short signatures, which are signatures with an L_2 norm less than a predetermined upper bound, β . The resulting secret key is those polynomials, (f, g, F, G) , and the public key is h , which is calculated through the polynomials (f, g) ($h = gf^{-1} \pmod{\phi \pmod{q}}$).

2) *Signing*: FALCON generates a signature by finding polynomials, (s_1, s_2) , such that it can find a point on the lattice, c , by solving $s_1 + s_2h = c \pmod{\phi \pmod{q}}$ using the public key, h . The lattice location, c , is the output of a hash function that transforms a byte message into a lattice location, making c unique to different signed messages. Then, s_2 is returned as the signature.

3) *Verification*: FALCON verifies a signature by computing the s_1 polynomial from the signature, s_2 , and the public key, h . Then, the signature is verified if the polynomials (derived s_1, s_2) are short. Although it is easy to find polynomials that solve $s_1 + s_2h = c \pmod{\phi \pmod{q}}$ through Gaussian elimination, it is difficult to find short polynomials that do so, and this constitutes the difficulty of the SIS problem.

4) *FALCON Public Key Recovery*: Another way to set up the FALCON system is to do it with public key recovery, which we will short-hand as pk-FALCON. In pk-FALCON, the verification step checks that the derived public key matches the original public key. This is done to further minimize the size of the signature and the public key. The signature generated by pk-FALCON would be both polynomials, (s_1, s_2) , and the public key is derived by solving $h_{derived} = (c - s_1)s_2^{-1} \pmod{\phi \pmod{q}}$. The signature is then verified if $h_{derived}$ matches the original public key.

B. Code Design

We aim to evaluate the performance slowdown and the additional storage requirements incurred by making Ethereum quantum-resistant. Specifically, we focus solely on the execution layer to analyze the performance impact of using FALCON instead of *Secp256k1*. To achieve this, we use existing FALCON libraries [14] and implement the public key recovery method described previously. The execution was simulated using a Rust-based implementation of the cryptographic scheme. We chose Rust for its speed, efficiency, and memory safety, which are important for a public blockchain. These attributes also underpin the development of *Reth* [21], a Rust implementation of the Ethereum execution layer.

C. Evaluation Metrics

Following our code design, we benchmark the performance of key generation, signature generation, and signature verification across three cryptographic schemes: *Secp256k1*, FALCON without public key recovery, and FALCON with

Cryptography Scheme	NIST Level	Implementation Details
Secp256k1	1*	C integration
FALCON512	1	512-byte FALCON
FALCON1024	5	1024-byte FALCON
C-FALCON512	1	C integration FALCON512
C-FALCON1024	5	C integration FALCON1024
pk-FALCON512	1	public key recovery FALCON512
pk-FALCON1024	5	public key recovery FALCON1024

TABLE I

SUMMARY TABLE OF THE CRYPTOGRAPHY SCHEMES INVESTIGATED.

*SECP256K1 IS NOT FORMALLY RECOGNIZED BY NIST UNDER A SECURITY LEVEL, BUT SIMILAR CURVES ARE DESIGNATED NIST LEVEL 1.

public key recovery. In addition, we include a C-based implementation of FALCON in the benchmark for comparison. The benchmarking process, implemented in Rust, takes 100 samples for each operation. We also record the storage requirements for each cryptographic method, specifically the space for storing the signature and either the public key or the address.

D. Implementation

The average speed of various cryptographic functions is benchmarked using a MacBook Air 2020 with an M1 chip (8-core, 3.2GHz), 8GB memory, 512GB SSD storage, and running macOS Sonoma 14.5. The cryptosystem storage requirements are also recorded. The benchmarking process involves generating 10 keys and producing 10 signatures per key pair. Alongside the average speed, the 95% confidence interval is determined. The cryptographic schemes investigated are summarized in Table I. FALCON512 is the closest to Secp256k1 in security level, so it will be discussed the most as the best comparable.

IV. RESULTS

A. Benchmark Speeds

1) **Key Generation: Key Generation Slow But Non-Impactful.** Among the three primary functions for digital signatures, key generation is the least critical, as it is typically performed only once per user and occurs off-chain. The key generation process for FALCON is significantly slower, ranging from 23,000 to 130,000 $\times$ slower than Secp256k1, as seen in Figure 2(a). Despite these disparities, the off-chain nature and one-time execution of key generation mitigate its impact on users.

2) **Signature Generation:** Signature generation is also performed off-chain, minimizing concerns about computational complexity and on-chain costs. Nevertheless, the speed of signature generation remains important as it directly affects user experience on the blockchain.

FALCON Signature Generation is over 26 $\times$ slower. The signature generation speed of Secp256k1 is approximately 22 μ s, whereas FALCON exhibits significantly slower performance (see Figure 2(b)). Among FALCON variants, C-FALCON512 is the fastest, requiring around 152 μ s, which is approximately 7 $\times$ slower than Secp256k1, but pk-FALCON512 took 28 $\times$ longer. Also, pk-FALCON is not much slower than FALCON.

3) **Signature Verification: pk-FALCON Signature Verification is over 3 $\times$ slower.** FALCON verification times are comparable to those of Secp256k1, taking approximately 24 μ s, while pk-FALCON verification is about 3 $\times$ slower than standard FALCON (see Figure 2(c)).

B. Benchmark Memory Storage Usage

1) **Cryptography Elements Storage:** For FALCON, the storage requirements for each element are significantly higher than for Secp256k1. The secret key for 512-byte FALCON implementations is 40 $\times$ larger than that of Secp256k1, but this is off-chain, so it is less of a concern. However, the public key may need to be stored on-chain, making its size a critical consideration. For FALCON512, the public key requires 897 bytes, which is 14 $\times$ larger than Secp256k1, and the 1024-byte version of the public key is twice as large as the 512-byte version. This highlights an advantage of the public key recovery version of FALCON, as only the address, a hash of the public key, will need to be stored on-chain. To ensure collision resistance, the recommended address or hash size is 32 bytes for FALCON512 and 64 bytes for FALCON1024.

Although public key recovery reduces the storage required, the signatures it generates are significantly larger. FALCON signatures require 666 bytes and 1280 bytes for FALCON512 and FALCON1024, respectively, while Secp256k1 requires only 65 bytes. FALCON signatures must store the header (1 byte), the salt (40 bytes), and the s_2 polynomial (625 bytes for FALCON512 and 1239 bytes for FALCON1024). In pk-FALCON, an additional polynomial must be stored, resulting in nearly double the signature size as in FALCON.

2) **On-Chain Storage:** FALCON's on-chain storage requirements are over 15 $\times$ bigger. Regardless of public key recovery, FALCON requires far more storage than Secp256k1. The lowest memory requirement is for pk-FALCON512, which needs 1323 bytes to store the sender's address and signature on-chain, and pk-FALCON is in general 15% smaller than FALCON. This is in contrast to Secp256k1, which requires 85 bytes. Overall, this makes pk-FALCON require more than 15 $\times$ the amount of storage for cryptography elements.

C. On-Chain Storage Cost

Given the estimates of the amount of additional storage taken up by FALCON as opposed to Secp256k1, this difference can be translated into additional gas costs. The most similar storage format for a larger transaction is the cost of nonzero call data bytes within a transaction, and that is priced at 16 gas per byte [22]. Using this, the estimated increases in gas cost can be seen in Table II.

pk-FALCON512 increases block size by 3.85 $\times$.

To estimate the impact on block size, the average transactions per block was found. It was done by dividing the daily total transactions by the number of seconds in a day by the average block time in seconds. Using data on September 5th, 2024 on Ycharts [23], it was found to be roughly 156 transactions. So if 156 transactions went from using Secp256k1 to pk-FALCON512, the block size would increase by $\frac{1323}{85} =$

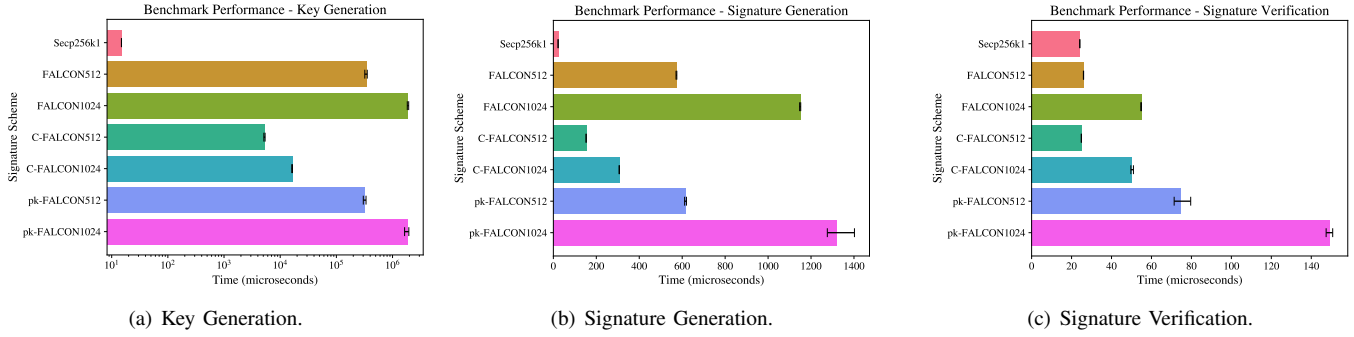


Fig. 2. Performance Comparison of Various Signature Schemes.

Cryptography Scheme	On-Chain Storage (bytes)	Gas Estimate
Secp256k1	85	1360
FALCON512	1563	25008
FALCON1024	3073	49168
pk-FALCON512	1323	21168
pk-FALCON1024	2583	41328

TABLE II

ESTIMATED GAS COST FOR STORING THE CRYPTOGRAPHIC ELEMENTS WITHIN THE SIGNATURE FOR VARIOUS CRYPTOGRAPHY SCHEMES.

156 $\times$, resulting in a final average block size of 260,772 bytes from 67,644 bytes, an increase of around 3.85 $\times$.

V. DISCUSSION

Future research on switching from elliptic curves to lattice-based cryptography can look to analyzing the increased gas cost associated with the more complicated mathematics required, as well as building out a fully functioning execution layer integrating FALCON.

A. Better Gas Estimate

As there is a way to obtain a general estimate of the increase in gas usage due to the additional storage requirements of FALCON, it would be helpful to compare that against the computational complexity of FALCON and the gas cost associated with it. Without having built an Ethereum node fully running on FALCON, another potential way to investigate run-time costs is to dive deep into the differences between the EVM-compiled opcodes for Secp256k1 and FALCON. Ethereum has a database of the gas cost of different opcodes [24], so it is possible to map out the EVM-compiled code of the verification methods for Secp256k1 and FALCON to see what the difference in gas cost would be. Key and signature generation are done off-chain, and only verification is done on-chain. Therefore, signature verification is the only function that needs to be checked in this way. However, this is notably not accurate, as it is difficult to estimate the number of loops that need to be done within the verification function and that will greatly impact the number of operations run.

B. Challenges of a FALCON-Ethereum

It would be best to build a complete node using only FALCON cryptography, but there are many challenges asso-

ciated with doing so. Primarily, the biggest challenge is that the consensus layer uses the BLS digital signature scheme, and that system uses elliptic curves and allows for signature aggregation. WE leave this replacement for future work.

Also, another challenge is the engineering difficulty of replacing the usage of Secp256k1 with FALCON as references to Secp256k1 are pervasive through the client code base. In Reth, the main usage of Secp256k1 is in the primitives crate [21], where it holds code that describes signature generation and public key recovery. However, most other crates refer back to Secp256k1 for various reasons, whether to store public keys or others.

VI. CONCLUSION

Ethereum is one of the most popular public blockchains, but it relies on the ECDSA, which can be broken by quantum computers. Upgrading from Secp256k1 to pk-FALCON512, the post-quantum cryptography scheme with the smallest signature and public key sizes compared to other post-quantum schemes, would result in 28 $\times$ slower signature generation, 3 $\times$ slower signature verification, and 15 $\times$ greater storage requirement. However, while the storage taken up by FALCON cryptography elements is much larger than Secp256k1, it only increases the average block size by 3.85 $\times$, suggesting that it may not be a big concern. However, this doesn't factor in the cost estimate of the additional storage required, nor the additional gas consumed by running the more computationally difficult algorithms. Overall, quantum computers may come sooner than expected, and it is important to have systems ready for when that happens. It's shown here that FALCON is a strong candidate for replacing ECDSA. Future research can look to build out a fully functioning execution layer using pk-FALCON512 and obtain more representative performance metrics from that implementation. Another option includes compiling the signature verification methods on EVM, and comparing the opcodes to provide an estimate of the difference in gas usage between FALCON and Secp256k1.

ACKNOWLEDGEMENTS

This work was supported in part by the Engineering and Physical Sciences Research Council (EPSRC) [grant number EP/X037010/1].

REFERENCES

- [1] Bitcoin Wiki contributors, "Secp256k1." Bitcoin Wiki, 2024. Accessed on August 29, 2024. pages 1
- [2] V. Buterin, "Ethereum: A next-generation smart contract and decentralized application platform." https://ethereum.org/content/whitepaper/whitepaper-pdf/Ethereum_Whitepaper_-_Buterin_2014.pdf, 2014. Accessed: [2024-08-29]. pages 1
- [3] K. E. Lauter and K. E. Stange, "The elliptic curve discrete logarithm problem and equivalent hard problems for elliptic divisibility sequences," in *Selected Areas in Cryptography* (R. M. Avanzi, L. Keliher, and F. Sica, eds.), (Berlin, Heidelberg), pp. 309–327, Springer Berlin Heidelberg, 2009. pages 1, 2
- [4] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," *SIAM J. Comput.*, vol. 26, p. 1484–1509, oct 1997. pages 1, 2
- [5] L. Chen, S. Jordan, Y.-K. Liu, D. Moody, R. Peralta, R. Perlner, and D. Smith-Tone, "Report on post-quantum cryptography," NIST Internal Report 8105, National Institute of Standards and Technology, April 2016. pages 1
- [6] NIST, "Migration to post-quantum cryptography." <https://www.nccoe.nist.gov/crypto-agility-considerations-migrating-post-quantum-cryptographic-algorithms>, 2024. Accessed: December 17, 2024. pages 1
- [7] Apple Security Research, "imessage with pq3: The new state of the art in quantum-secure messaging at scale." Online, February 2024. Apple Security Research Blog. pages 1
- [8] Zoom Video Communications, Inc., "Zoom bolsters security offering with the inclusion of post-quantum end-to-end encryption in zoom workplace." Online, 2024. Zoom News Blog. pages 1
- [9] M. Swayne, "Ethereum prepares for quantum-resistant future amid security push," Oct. 2024. Accessed: 2025-01-10. pages 1
- [10] Z. Yang, H. Alfauri, B. Farkiani, R. Jain, R. Di Pietro, and A. Erbad, "A survey and comparison of post-quantum and quantum blockchains," *IEEE Communications Surveys & Tutorials*, 2023. pages 1
- [11] A.-T. Ciulei, M.-C. Crețu, and E. Simion, "Preparation for post-quantum era: a survey about blockchain schemes from a post-quantum perspective," *Cryptology ePrint Archive*, 2022. pages 1
- [12] T. M. Fernandez-Carames and P. Fraga-Lamas, "Towards post-quantum blockchain: A review on blockchain cryptography resistant to quantum computing attacks," *IEEE access*, vol. 8, pp. 21091–21116, 2020. pages 1
- [13] P.-A. Fouque, J. Hoffstein, P. Kirchner, V. Lyubashevsky, T. Pornin, T. Prest, T. Ricosset, G. Seiler, W. Whyte, Z. Zhang, *et al.*, "Falcon: Fast-fourier lattice-based compact signatures over ntru," *Submission to the NIST's post-quantum cryptography standardization process*, vol. 36, no. 5, pp. 1–75, 2018. pages 1
- [14] P.-A. Fouque, J. Hoffstein, P. Kirchner, V. Lyubashevsky, T. Pornin, T. Prest, T. Ricosset, G. Seiler, W. Whyte, and Z. Zhang, "Falcon: Fast-fourier lattice-based compact signatures over ntru," specification, NIST, January 2020. Accessed on August 29, 2024. pages 1, 2
- [15] M. E. Hellman, "An overview of public key cryptography," *IEEE Communications Magazine*, vol. 16, November 1978. Reprinted in 50th Anniversary Commemorative Issue, May 2002. pages 1
- [16] Ethereum Foundation, "Networking layer," July 2024. Accessed on August 29, 2024. pages 2
- [17] S. Bai and R. P. Brent, "On the efficiency of pollard's rho method for discrete logarithms," in *Proceedings of the Fourteenth Symposium on Computing: The Australasian Theory - Volume 77*, CATS '08, (AUS), p. 125–131, Australian Computer Society, Inc., 2008. pages 2
- [18] National Institute of Standards and Technology, "Nist announces first four quantum-resistant cryptographic algorithms," July 2022. News release from the U.S. Department of Commerce's National Institute of Standards and Technology. pages 2
- [19] National Institute of Standards and Technology, "Nist releases first 3 finalized post-quantum encryption standards." Online, August 2024. NIST News Release. pages 2
- [20] D. Micciancio, "The sis problem and cryptographic applications," January 2020. Lecture slides from Simons Institute for the Theory of Computing. pages 2
- [21] Reth Team, "Reth book," 2024. Documentation for Reth users and developers. pages 2, 4
- [22] T. Wahrstätter and V. Buterin, "Eip-7623: Increase calldata cost," February 2024. Ethereum Improvement Proposals, no. 7623. pages 3
- [23] YCharts, "Economic indicators," September 2024. Accessed: September 6, 2024. pages 3
- [24] Ethereum Foundation, "Opcodes for the evm." <https://ethereum.org/en/developers/docs/evm/opcodes/>, 2024. Accessed on [2024-08-29]. pages 4